

Post Interview Task

Pranav TR

June 2026

1 Introduction

This paper goes over a probabilistic method for crypt-analyzing an N -key Crypto-system. It introduces a time-memory trade-off bridging exhaustive search and table lookup, allowing a key to be found in $N^{2/3}$ operations using $N^{2/3}$ words of memory.

2 Defining the One-Way Function

The technique relies on a chosen plaintext attack. Let S_K be the enciphering operation under key K . For a fixed, known plaintext P_0 , the intercepted ciphertext is:

$$C_0 = S_K(P_0)$$

To build the search space, a simple reduction function R (e.g., dropping bits) is applied to the 64-bit ciphertext to map it back to a 56-bit key space. This defines our core one-way function f :

$$f(K) = R[S_K(P_0)]$$

Finding K from $f(K)$ is equivalent to successfully crypt-analyzing the system.

3 Precomputation and Chain Generation

The precomputation phase builds a matrix of sequences (chains) to cover the key space. For m starting points, we define the starting point of the i -th chain as a random key:

$$SP_i = X_{i0}$$

Each chain is generated by iteratively applying the function f for t steps:

$$X_{ij} = f(X_{i,j-1}), \quad 1 \leq j \leq t$$

To save memory, all intermediate values are discarded. Only the starting points SP_i and the final endpoints EP_i are stored. The endpoint is simply the function applied t times:

$$EP_i = f^t(SP_i)$$

4 The Cryptanalysis Phase

Given the intercepted ciphertext C_0 , the cryptanalyst applies the reduction function R to find the first sequence value Y_1 :

$$Y_1 = R(C_0) = f(K)$$

The analyst checks if Y_1 matches any stored endpoint EP_i . If it does, the key K is likely the predecessor in that chain ($X_{i,t-1}$). If it does not match, the cryptanalyst iterates the function to look further down the theoretical chain:

$$Y_2 = f(Y_1)$$

This iteration continues ($Y_3 = f(Y_2)$, etc.) until an endpoint is matched, at which point the chain is regenerated from SP_i to uncover K .

5 Complexity and Probability of Success

If overlaps in the chains are negligible, a single table of m chains of length t covers mt unique keys. The probability of success $P(S)$ for a key chosen uniformly from N possibilities is bounded by:

$$P(S) \approx \frac{mt}{N}$$

To optimize the overall trade-off, we balance the memory required ($M \approx m$) and the time required for the search ($T \approx t^2$). The mathematical curve is optimized when $mt^2 = N$. Setting $m = t = N^{1/3}$ yields the classic complexity result:

$$M = T = N^{2/3}$$